

**Inteligencia de negocios para el sector vivienda: análisis del negocio, implementación del
ETL y construcción del DataMart de colocaciones Mivivienda 2024**

Daniel Alonso Correa Chanta

Jose Carlo Castro Franco

Universidad Tecnológica del Perú

Inteligencia de Negocios

Docente: Oscar Eduardo Balcazar Chumacero

Piura, Perú

2026

Resumen

El presente trabajo desarrolla de manera acumulativa el análisis del negocio y la implementación física de un DataMart para las colocaciones de créditos Mivivienda durante 2024. La primera parte define el problema, los usuarios, los indicadores clave de rendimiento, la fuente de datos y el modelo dimensional. La segunda parte implementa la base de datos, el proceso de extracción, transformación y carga, las cargas inicial e incremental, las pruebas y un dashboard web. La fuente contiene 13,507 filas; después de eliminar 4,160 filas vacías y 11 duplicados exactos se cargaron 9,336 hechos. Las pruebas verificaron integridad referencial, consistencia, calidad, rendimiento y coincidencia con la fuente. La solución utiliza Python, MySQL y Flask, y permite consultar indicadores por tiempo, producto, geografía, institución financiera y plazo.

Palabras clave: inteligencia de negocios, ETL, DataMart, modelo estrella, MySQL, Python.

Inteligencia de negocios para el sector vivienda: análisis e implementación del DataMart

Mivivienda 2024

El Fondo MIVIVIENDA S.A. promueve el acceso al financiamiento para la adquisición, construcción y mejoramiento de viviendas. Para este trabajo se utiliza un conjunto de datos público con las colocaciones realizadas durante 2024. Aunque la fuente contiene información suficiente para calcular indicadores, su estructura plana dificulta el análisis repetido por tiempo, producto, ubicación e institución financiera.

El modelado dimensional organiza los procesos de negocio alrededor de hechos medibles y dimensiones descriptivas, facilitando las consultas analíticas y la comprensión de los datos (Kimball & Ross, 2013). Por ello, el trabajo integra el análisis realizado en el Avance 1 con la implementación solicitada en el Avance 2. El documento no presenta dos informes independientes: conserva las decisiones conceptuales iniciales y muestra cómo fueron materializadas y verificadas.

El objetivo general es desarrollar la integración, transformación y carga de datos para construir físicamente el DataMart de colocaciones Mivivienda. Los objetivos específicos son implementar tablas, relaciones, índices y restricciones; desarrollar las etapas Extract, Transform y Load; realizar cargas inicial e incremental; validar integridad, consistencia, rendimiento y calidad; y presentar evidencias técnicas y visuales.

Parte I: Análisis y definición del negocio

Definición del problema de negocio

La necesidad principal consiste en convertir una tabla plana en una estructura analítica que permita responder preguntas de gestión de forma rápida y consistente. El problema no es la ausencia de información, sino la falta de una organización orientada al análisis. El DataMart

debe permitir conocer cuánto se colocó, cuándo se colocó, mediante qué producto, en qué ubicación y a través de qué institución financiera.

Tabla 1

Usuarios finales y necesidades de información

Usuario	Necesidad principal
Gerencia general	Revisar KPIs, montos y evolución mensual.
Analistas de negocio	Comparar productos, tasas, montos y plazos.
Área de riesgo	Evaluar tasas, plazos y concentración.
Equipo comercial	Identificar las IFI con mayor colocación.
Planificación	Analizar tendencias y concentración geográfica.

Nota. Elaboración propia a partir de los requerimientos del negocio.

Objetivos estratégicos e indicadores

Tabla 2

Objetivos estratégicos y KPIs

Objetivo	Indicadores
Analizar colocaciones	Monto total, cantidad, promedio y evolución mensual.
Analizar productos	Cantidad, monto y tasa promedio por producto.
Analizar IFI	Ranking por monto y participación por tipo de IFI.
Analizar geografía	Créditos por departamento, provincia y distrito.
Analizar condiciones	Plazo, tasa y relación cuota inicial/valor.

Proceso de negocio y alcance

El proceso principal es el desembolso de un crédito Mivivienda. Cada fila válida representa un crédito desembolsado por una institución financiera, en una fecha y ubicación determinadas. En consecuencia, el grano de la tabla de hechos es un registro por crédito desembolsado.

Tabla 3

Alcance del DataMart

Incluye	No incluye
Créditos desembolsados en 2024	Solicitudes rechazadas o en evaluación
Productos NMIV, FCTP y S-CRC	Datos personales de beneficiarios
Análisis geográfico e institucional	Morosidad, recuperaciones o castigos
Montos, tasa, plazo y valor	Rentabilidad interna de las IFI

Fuente de datos

La fuente corresponde al archivo Colocaciones de los Créditos Mivivienda 2024, publicado por Fondo MIVIVIENDA S.A. (2025). El archivo utiliza punto y coma como separador y contiene 14 columnas. El CSV funciona como sistema origen y se carga primero a una tabla staging antes de alimentar el modelo dimensional.

Tabla 4

Inventario de la fuente

Atributo	Detalle
Formato	CSV separado por punto y coma
Cobertura temporal	3 de enero al 30 de diciembre de 2024
Fecha de corte	8 de enero de 2025
Filas físicas	13,507
Filas válidas no vacías	9,347
Columnas	14

Nota. Los conteos fueron verificados directamente sobre el archivo entregado.

Tabla 5

Diccionario de la tabla origen

Campo	Descripción
FECHA_DESEMBOLSO	Fecha del desembolso en formato aaaammdd.
PRODUCTO	Código NMIV, FCTP o S-CRC.
DEPARTAMENTO/PROVINCIA/DISTRITO	Ubicación de la vivienda.
UBIGEO	Código geográfico de seis dígitos.
IFI/TIPO_IFI	Institución financiera y clasificación.
MONTO_CREDITO	Monto del crédito desembolsado.
MONTO_CUOTA_INICIAL	Cuota inicial aportada.
PLAZOS	Plazo del crédito en meses.
TASA	Tasa de interés anual.
MONTO_VALOR_VIVIENDA	Valor total de la vivienda.
FECHA_CORTE	Fecha de corte del conjunto de datos.

Problemas de calidad identificados

Tabla 6

Problemas de calidad y tratamiento

Problema	Tratamiento ETL
4,160 filas completamente vacías	Eliminar antes de convertir tipos.

Problema	Tratamiento ETL
Fechas numéricas aaaammdd	Convertir a DATE.
UBIGEO susceptible de conversión numérica	Mantener como CHAR(6).
Textos con formatos variables	Aplicar TRIM, espacios uniformes y mayúsculas.
Datos repetidos	Eliminar duplicados exactos y generar hash.
Redundancia geográfica e institucional	Separar en dimensiones.

Requerimientos analíticos

Tabla 7

Preguntas de negocio

N.º	Pregunta
1	¿Cuál fue el monto total desembolsado por mes?
2	¿Qué producto tuvo mayor cantidad de créditos?
3	¿Qué departamentos concentran las colocaciones?
4	¿Cuál es el ranking de IFI por monto colocado?
5	¿Cómo varía la tasa por producto y tipo de IFI?
6	¿Cuántos créditos pertenecen a cada categoría de plazo?
7	¿Qué trimestre registró el mayor monto?

Modelo conceptual inicial

Se seleccionó un modelo estrella porque ofrece una estructura simple para consultas agregadas y es coherente con una fuente desnormalizada. El modelo considera una tabla de hechos central y cinco dimensiones: tiempo, geografía, producto, institución financiera y plazo. Esta decisión sigue los principios de modelado dimensional propuestos por Kimball y Ross (2013).

Tabla 8

Dimensiones del modelo conceptual

Dimensión	Atributos principales
DIM_TIEMPO	Fecha, año, semestre, trimestre, mes y día.
DIM_GEOGRAFIA	Ubigeo, departamento, provincia y distrito.
DIM_PRODUCTO	Código, nombre y descripción.
DIM_IFI	Nombre y tipo de institución.
DIM_PLAZO	Meses, años, rango y categoría.

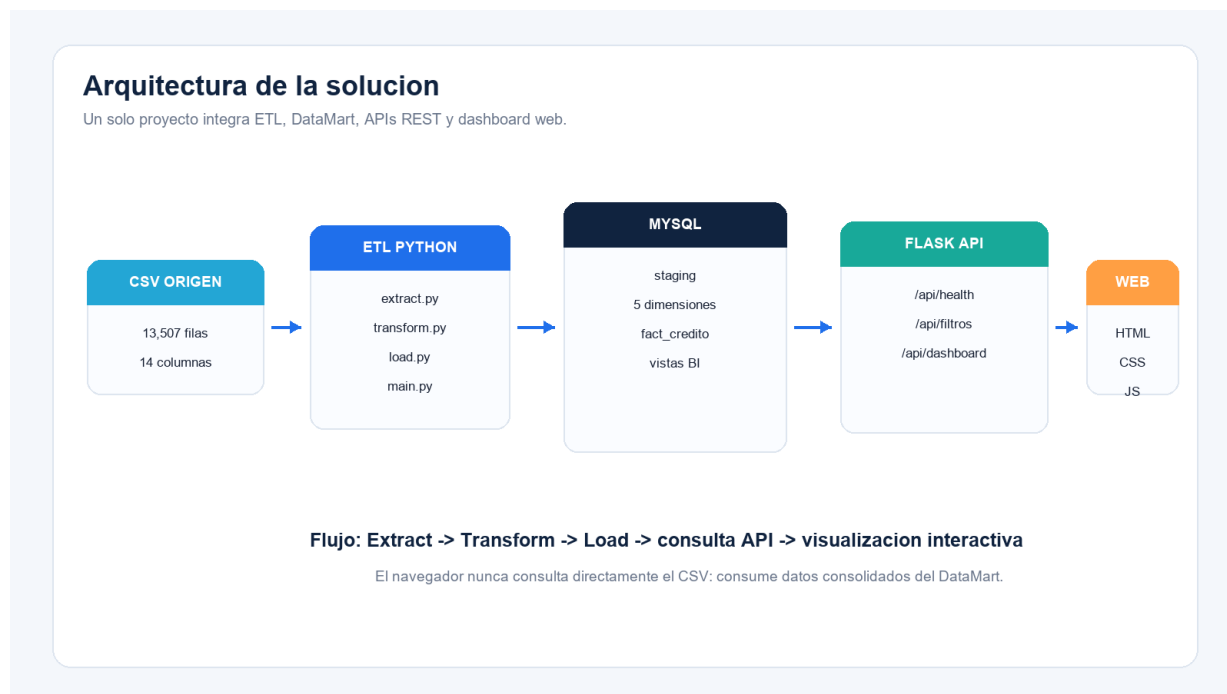
Parte II: Implementación del ETL y construcción del DataMart

Metodología y herramientas

El enunciado presenta PostgreSQL, SQL Server y Oracle como herramientas posibles, y menciona SSIS, Pentaho y Talend como alternativas ETL. La expresión herramientas posibles no restringe la implementación a dichas opciones. Se utilizó MySQL porque el Avance 1 ya lo establecía como destino del DataMart, y Python con pandas porque permite implementar explícitamente las etapas de extracción, transformación y carga. MySQL proporciona claves primarias, foráneas, índices, restricciones y vistas; pandas ofrece lectura, limpieza y transformación tabular (Oracle, 2026; pandas development team, 2026).

Figura 1

Arquitectura de la solución implementada



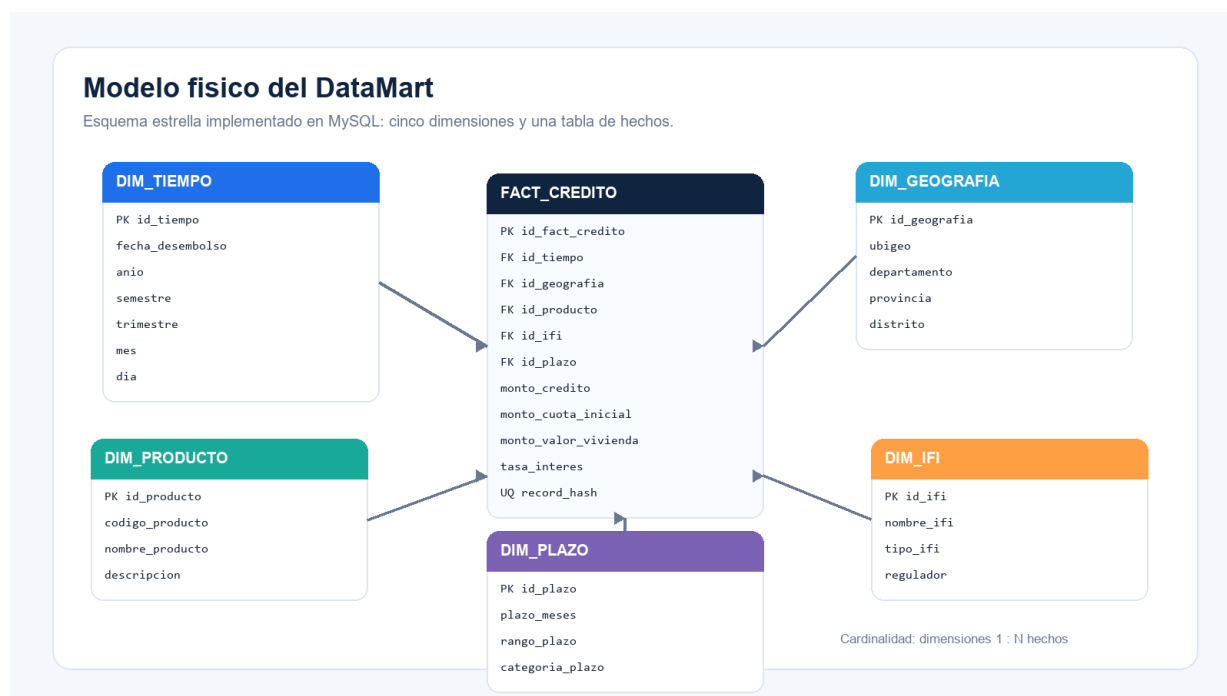
Nota. El navegador consume APIs Flask; el CSV solo es procesado por el ETL.

Implementación de la base de datos

La base mivivienda_dm se crea mediante setup_database.py, que ejecuta los scripts 01_staging.sql y 02_datamart.sql. Se implementaron staging, cinco dimensiones, la tabla fact_credito, la tabla de auditoría etl_ejecucion y dos vistas analíticas.

Figura 2

Modelo físico del DataMart



Nota. Las relaciones son de uno a muchos desde cada dimensión hacia fact_credito.

Tabla 9

Objetos físicos implementados

Objeto	Función
stg_colocaciones_mivivienda	Recibe el lote transformado.
dim_tiempo	Organiza la jerarquía temporal.
dim_geografía	Almacena ubicaciones únicas por ubigeo.
dim_producto	Describe los productos.
dim_ifi	Describe las instituciones financieras.
dim_plazo	Clasifica el plazo en meses y categorías.
fact_credito	Almacena medidas y claves foráneas.
etl_ejecucion	Registra métricas de cada carga.
vw_creditos_analitica	Integra hechos y dimensiones para BI.

Relaciones, índices y restricciones

- Cinco claves foráneas conectan fact_credito con las dimensiones.
- Las claves subrogadas son autoincrementales.
- record_hash posee una restricción UNIQUE.
- Se crearon índices para fecha, ubigeo, producto y claves foráneas.
- Las restricciones CHECK controlan montos, tasas, plazos y cantidad.
- Las columnas críticas utilizan NOT NULL.

Desarrollo del proceso ETL

Extract

extract.py lee el archivo con separador punto y coma, codificación UTF-8 y todas las columnas como texto. También valida que estén presentes las 14 columnas esperadas. La lectura conserva las filas vacías para poder contarlas y documentar su eliminación.

```
df = pd.read_csv(  
    path,  
    sep=";",  
    dtype="string",  
    encoding="utf-8-sig",  
    skip_blank_lines=False,  
)
```

Transform

transform.py reemplaza cadenas vacías por valores nulos, elimina filas vacías, normaliza textos, convierte fechas y campos numéricos, valida reglas de negocio y elimina duplicados. Después genera record_hash con SHA-256 sobre los 14 valores normalizados. Esta clave técnica compensa la ausencia de un identificador de transacción en la fuente.

Load

load.py carga staging, dimensiones y hechos en ese orden. Las dimensiones se insertan antes de fact_credito para disponer de las claves subrogadas. La integración se realiza mediante

JOIN entre staging y las cinco dimensiones. Finalmente, record_execution registra el modo, los conteos y el estado de la ejecución.

Carga inicial e incremental

La carga inicial reinicia dimensiones y hechos y construye el DataMart desde cero. La carga incremental mantiene la información existente y utiliza record_hash para insertar únicamente filas nuevas.

Figura 3

Ejecución de las cargas inicial e incremental

Evidencia de ejecución ETL
Comparacion de carga inicial e incremental registrada en MySQL.

```
PowerShell - ETL Mivivienda

> python -m etl.main --mode initial
=== INICIO ETL MIVIVIENDA (INITIAL) ===
[EXTRACT] Filas leídas: 13,507
[TRANSFORM] Vacías: 4,160 | Duplicadas: 11 | Finales: 9,336
[LOAD] Hechos nuevos insertados: 9,336
=== ETL FINALIZADO - ESTADO OK ===

> python -m etl.main --mode incremental
=== INICIO ETL MIVIVIENDA (INCREMENTAL) ===
[EXTRACT] Filas leídas: 13,507
[TRANSFORM] Vacías: 4,160 | Duplicadas: 11 | Finales: 9,336
[LOAD] Hechos nuevos insertados: 0
=== ETL FINALIZADO - ESTADO OK ===
```

Nota. La carga inicial insertó 9,336 hechos y la incremental posterior insertó cero.

Tabla 10

Resultados de carga

Modo	Leídas	Vacías	Duplicados	Transformadas	Insertadas
Inicial	13,507	4,160	11	9,336	9,336
Incremental	13,507	4,160	11	9,336	0

Validación y pruebas

Integridad de datos

Se comprobó que las 9,336 filas de staging tienen correspondencia en fact_credito, que todos los hashes son únicos, que no existen claves foráneas nulas y que no existen hechos huérfanos. Las restricciones de MySQL complementan estas consultas al impedir valores incompatibles.

Figura 4

Resultados de integridad y calidad



Nota. Los conteos fueron obtenidos directamente de MySQL.

Consistencia y comparación con el origen

Tabla 11

Conciliación entre origen y DataMart

Concepto	Cálculo	Resultado
Filas físicas	-	13,507
Menos filas vacías	13,507 - 4,160	9,347
Menos duplicados	9,347 - 11	9,336

Concepto	Cálculo	Resultado
Hechos cargados	COUNT(fact_credito)	9,336
Diferencia	Esperado - cargado	0

Calidad de información

La calidad se evaluó mediante nulos críticos, montos positivos, fechas válidas, formato de ubigeo, duplicados y consistencia de claves. El resultado fue cero claves foráneas nulas, cero montos inválidos y 9,336 hashes únicos.

Rendimiento

Se ejecutó 20 veces la consulta completa del servicio del dashboard, que incluye KPIs, serie mensual, productos, departamentos, instituciones y 100 registros de detalle. El promedio fue 186.91 ms, la mediana 181.67 ms, el mínimo 171.93 ms y el máximo 227.49 ms. Para el volumen académico de 9,336 hechos, los tiempos son adecuados para una interacción web local.

Tabla 12

Prueba de rendimiento del dashboard

Ejecuciones	Promedio	Mediana	Mínimo	Máximo
20	186.91 ms	181.67 ms	171.93 ms	227.49 ms

Nota. Medición local realizada sobre DashboardService conectado a MySQL.

Pruebas automatizadas

Figura 5

Ejecución de pruebas automatizadas



```

Pruebas automatizadas
Pruebas unitarias del ETL y pruebas de integración de las APIs Flask.

> python -m unittest discover -s tests -v

test_transform_rejects_invalid_business_values ... ok
test_transform_removes_empty_and_duplicate_rows ... ok
test_dashboard_returns_kpis_and_filtered_data ... ok
test_health_connects_to_database ... ok
test_index_serves_dashboard ... ok

-----
Ran 5 tests

OK

```

Nota. Las cinco pruebas finalizaron correctamente.

Tabla 13

Cobertura de pruebas automatizadas

Prueba	Resultado
Eliminación de filas vacías	Correcto
Eliminación de duplicados	Correcto
Rechazo de valores de negocio inválidos	Correcto
Conexión de la API con MySQL	Correcto
Aplicación de filtros LIMA y NMIV	Correcto
Entrega de la página HTML	Correcto

Verificación de KPIs

Tabla 14

Indicadores generales del DataMart

Indicador	Resultado
Cantidad de créditos	9,336
Monto total	S/ 1,695,883,888.67
Monto promedio	S/ 181,649.95
Tasa promedio	10.03%

Indicador	Resultado
Tasa mínima y máxima	5.9900% y 23.8900%
Plazo mínimo y máximo	14 y 300 meses

Tabla 15*Resultados por producto*

Producto	Créditos	Monto total
NMIV	8,565	S/ 1,557,092,996.98
S-CRC	352	S/ 121,318,382.44
FCTP	419	S/ 17,472,509.25

Tabla 16*Principales departamentos*

Departamento	Créditos	Monto total
LIMA	5,789	S/ 1,236,814,750.52
LAMBAYEQUE	627	S/ 80,598,442.85
CALLAO	396	S/ 79,108,207.54
PIURA	633	S/ 68,120,292.91
LA LIBERTAD	474	S/ 66,845,277.69

Dashboard web

La capa de presentación se implementó con Flask, HTML, CSS, JavaScript y Chart.js.

Flask sirve la página y expone las rutas /api/health, /api/filtros y /api/dashboard. JavaScript utiliza fetch para solicitar JSON y actualizar tarjetas, gráficos y tabla. Flask es un framework flexible que permite organizar controladores y servicios sin exigir una estructura específica (Pallets Projects, 2026).

Figura 6

Dashboard de colocaciones Mivivienda 2024



Nota. La interfaz consulta la vista analítica de MySQL y permite filtrar por departamento, producto y tipo de IFI.

Evidencias técnicas y cumplimiento

Tabla 17

Matriz de cumplimiento de los requisitos del Avance 2

Requisito	Implementación y evidencia	Estado
Creación de tablas	01_staging.sql y 02_datamart.sql	Cumplido
Relaciones	Cinco claves foráneas en fact_credito	Cumplido
Índices	Índices de staging, hechos y vistas	Cumplido
Restricciones	PK, FK, UNIQUE, NOT NULL y CHECK	Cumplido
Extract	etl/extract.py	Cumplido
Transform	etl/transform.py	Cumplido
Load	etl/load.py y etl/main.py	Cumplido
Limpieza y formatos	Filas vacías, fechas, textos y números	Cumplido
Duplicados	drop_duplicates y record_hash	Cumplido
Integración	Cinco dimensiones y una tabla de hechos	Cumplido
Carga inicial	9,336 filas insertadas	Cumplido
Carga incremental	0 duplicados en segunda carga	Cumplido
Integridad	Cero FK nulas y cero hechos huérfanos	Cumplido
Consistencia	Diferencia origen/DataMart igual a cero	Cumplido

Requisito	Implementación y evidencia	Estado
Rendimiento	20 consultas; promedio 186.91 ms	Cumplido
Calidad	Cero montos inválidos y hashes únicos	Cumplido
Verificación de KPIs	04 _consultas_kpi.sql y dashboard	Cumplido
Evidencias	Figuras 1 a 6, tablas y scripts	Cumplido

Nota. La matriz relaciona cada actividad solicitada con un artefacto verificable del proyecto.

Discusión

La implementación confirma que una fuente compuesta por una sola tabla puede transformarse en un DataMart útil cuando se define correctamente el grano y se separan los contextos descriptivos. El modelo estrella reduce la repetición lógica en las consultas y permite que los indicadores se calculen sobre una estructura estable.

La ausencia de un identificador único en la fuente constituyó la principal limitación técnica. El hash del contenido permite idempotencia frente al mismo archivo; sin embargo, si dos operaciones reales distintas tuvieran exactamente los mismos 14 valores, serían interpretadas como duplicadas. En una implementación institucional sería preferible que la fuente proporcione un identificador transaccional.

La arquitectura Flask facilita la explicación del proyecto porque separa las rutas HTTP, el servicio de consultas y el frontend. No obstante, el objetivo central del avance es el ETL y el DataMart; el dashboard actúa como evidencia de explotación y no reemplaza las validaciones del modelo.

Conclusiones

1. El análisis del Avance 1 se materializó en un modelo estrella físico con cinco dimensiones y una tabla de hechos.
2. El proceso ETL trató filas vacías, formatos, textos, valores numéricos y duplicados, y cargó 9,336 hechos válidos.

3. La carga incremental fue idempotente para el archivo evaluado y no insertó registros repetidos.
4. Las pruebas demostraron integridad referencial, consistencia con la fuente, calidad de información y tiempos de consulta adecuados.
5. Los KPIs y el dashboard responden las preguntas de negocio definidas inicialmente.
6. Las evidencias y la matriz de cumplimiento demuestran que se cubrieron las actividades y entregables solicitados para el Avance 2.

Referencias

- Fondo MIVIVIENDA S.A. (2025). Colocaciones de los créditos Mivivienda 2024 [Conjunto de datos]. Plataforma Nacional de Datos Abiertos.
- Kimball, R., & Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3rd ed.). Wiley.
- Oracle. (2026). MySQL reference manual. <https://dev.mysql.com/doc/>
- Pallets Projects. (2026). Flask documentation. <https://flask.palletsprojects.com/>
- pandas development team. (2026). pandas documentation. <https://pandas.pydata.org/docs/>
- Universidad Tecnológica del Perú. (2026). Guía de Avance 2: Implementación del ETL y construcción del DataMart [Material del curso].

Apéndice A

Archivos principales del proyecto

Tabla 18

Archivos y responsabilidades

Archivo	Responsabilidad
etl/conexion.py	Configuración y conexiones MySQL.
etl/setup_database.py	Creación automática de la base.
etl/extract.py	Lectura y validación del CSV.

Archivo	Responsabilidad
etl/transform.py	Limpieza, conversión, reglas y hash.
etl/load.py	Carga de staging, dimensiones y hechos.
etl/main.py	Orquestación inicial e incremental.
web/app.py	Rutas Flask y respuestas JSON/HTML.
web/services/dashboard_service.py	Consultas analíticas y filtros.
sql/03_validaciones.sql	Pruebas de integridad y conciliación.
sql/04_consultas_kpi.sql	Consultas de indicadores.

Apéndice B

Comandos de ejecución

```

.\.venv\Scripts\python.exe -m etl.setup_database
.\.venv\Scripts\python.exe -m etl.main --mode initial
.\.venv\Scripts\python.exe -m etl.main --mode incremental
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
.\.venv\Scripts\python.exe -m web.app

```

Dashboard: <http://localhost:5000>